

Universidad San Carlos de Guatemala

Facultad de Ingeniería

Practicas Iniciales

Manual de uso de Docker

Grupo No.8

No.	Carnet	Nombre
1	202503661	Juan Luis Galicia Mazariegos
2	202505069	José Carlos González López
3	202502811	Wagner Maximiliano Ley Monroy
4	202500165	Daniel Estuardo Chicoj Bolaños
5	202406099	Guillermo Joaquín González Bámaca
6	202504673	José Daniel Tejaxún Xicón
7	202200257	Anderson Daniel Durán Herrera

Supervisora 1: Astrid Vanesa Kim Ortiz

Supervisora 2: Mayelin Lucía Quevedo Camey

15/08/2026

DIDT DESARROLLO DE TRANSFERENCIA TECNOLÓGICA | ECYS

Manual de Uso de Docker

1. Introducción: ¿Qué es Docker?

Docker es una plataforma de código abierto que permite empaquetar una aplicación junto con **todas sus dependencias** (librerías, runtime, configuración) dentro de una unidad aislada llamada **contenedor**. Ese contenedor se ejecuta igual en cualquier máquina que tenga Docker instalado, sin importar el sistema operativo o las versiones instaladas en la máquina anfitriona.

Esto resuelve el clásico problema de *"en mi máquina sí funciona"*: si la aplicación corre dentro del contenedor, correrá idéntica en la computadora del compañero, en el servidor de la universidad o en la nube.

1.1 Diferencia con una máquina virtual

Aspecto	**Máquina Virtual**	**Contenedor Docker**
Qué virtualiza	Hardware completo	Solo el espacio de usuario
Sistema operativo	Cada VM lleva su propio SO completo	Comparten el kernel del anfitrión
Tamaño típico	Varios GB	Decenas o cientos de MB
Tiempo de arranque	Minutos	Segundos
Consumo de recursos	Alto	Bajo

1.2 Conceptos fundamentales

Estos cuatro conceptos son la base de todo lo demás:

- **Imagen (image):** una plantilla inmutable de solo lectura que contiene la aplicación y todo lo que necesita para correr. Es como el "molde" o el instalador. Ejemplo: `tonelito762/pi-server:latest`.
- **Contenedor (container):** una instancia en ejecución de una imagen. Si la imagen es el model el container es el pastel. Es el proceso vivo. De una misma imagen se pueden levantar muchos contenedores simultáneos. Ejemplo: `docker-pi-server-1`.
- **Dockerfile:** un archivo de texto con las instrucciones para **construir** una imagen. Son las medidas del molde.
- **DockerHub:** el repositorio público en línea donde se almacenan y comparten imágenes. Es el "GitHub de las imágenes de Docker".

La relación entre ellos es secuencial:

```
Dockerfile  --(docker build)-->  Imagen  --(docker run)-->  Contenedor
                                   |
                                   +--(docker push)--> DockerHub
```

2. ¿Qué es un Dockerfile?

Un **Dockerfile** es un archivo de texto plano (sin extensión) que contiene la lista de instrucciones que Docker ejecuta, en orden, para construir una imagen. Cada instrucción genera una **capa** (layer) que Docker guarda en caché: si una capa no cambia entre dos construcciones, Docker la reutiliza en lugar de rehacerla, lo que acelera enormemente los builds posteriores.

3.1 Instrucciones principales

Instrucción	Para qué sirve
`FROM`	Define la imagen base sobre la que se construye. Siempre es la primera línea.
`WORKDIR`	Establece el directorio de trabajo dentro del contenedor. Lo crea si no existe.
`COPY`	Copia archivos desde la máquina anfitriona hacia la imagen.
`RUN`	Ejecuta un comando **durante la construcción** de la imagen (por ejemplo, instalar dependencias).
`ARG`	Define una variable disponible **solo durante el build**. Se puede pasar con `--build-arg`.
`ENV`	Define una variable de entorno que persiste dentro de la imagen.
`EXPOSE`	Documenta el puerto en el que escucha la aplicación. **No publica el puerto**; eso lo hace `-p` al correr.
`CMD`	Comando que se ejecuta **cada vez que arranca un contenedor** a partir de la imagen.

Es importante entender la diferencia entre `RUN` y `CMD`: `RUN` ocurre una sola vez, mientras se construye la imagen. `CMD` ocurre cada vez que se levanta un contenedor.

3.2 Ejemplo de dockerfile de una app de servidor javascript

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --omit=dev

COPY src ./src

EXPOSE 4000
CMD ["node", "src/index.js"]
```

Explicación línea por línea:

- `FROM node:20-alpine` — parte de la imagen oficial de Node.js versión 20 sobre Alpine Linux, una distribución minimalista de apenas unos MB, lo que mantiene la imagen final ligera.
- `WORKDIR /app` — todos los comandos siguientes se ejecutan dentro de `/app`.
- `COPY package*.json ./` — copia únicamente `package.json` y `package-lock.json` **antes** que el código fuente. Esto es una optimización de caché: como estos archivos cambian con poca frecuencia, la capa del `npm install` se reutiliza mientras no se modifiquen las dependencias.
- `RUN npm install --omit=dev` — instala las dependencias de producción, omitiendo las de desarrollo para reducir el tamaño de la imagen.
- `COPY src ./src` — ahora sí copia el código fuente. Al ir después del `install`, un cambio en el código no obliga a reinstalar todas las dependencias.
- `EXPOSE 4000` — documenta que la aplicación escucha en el puerto 4000.
- `CMD ["node", "src/index.js"]` — arranca el servidor Express cada vez que se levanta el contenedor.

3.3 Dockerfile de una aplicación cliente (multi-stage build)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG VITE_API_URL=http://localhost:4000
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Este Dockerfile usa la técnica de **construcción en múltiples etapas (multi-stage build)**, identificable por tener dos instrucciones `FROM`:

Etapa 1 (`AS build`) — utiliza Node.js para compilar la aplicación de React. El comando `npm run build` transforma el código JSX en archivos estáticos de HTML, CSS y JavaScript, que quedan en la carpeta `dist/`.

Las líneas `ARG` y `ENV` merecen atención especial. Vite (la herramienta de compilación de React) **reemplaza la variable `VITE_API_URL` por su valor literal durante la compilación**, dejándola fija dentro del JavaScript generado. Por eso el valor debe estar disponible en el momento del build y no en el momento de ejecución.

Etapa 2 — comienza desde cero con `nginx:alpine`, un servidor HTTP ligero. La instrucción `COPY --from=build` toma **solo la carpeta `dist/`** que produjo la etapa anterior y la coloca en el directorio que Nginx sirve por defecto.

La ventaja es el tamaño: la imagen final contiene únicamente Nginx y los archivos estáticos. Node.js, npm, `node_modules` y el código fuente de React quedan descartados junto con la primera etapa. El resultado son **93 MB** en lugar de los varios cientos de MB que ocuparía incluir todo el entorno de Node.

3.4 El archivo .dockerignore

Funciona de forma similar a `.gitignore`: indica qué archivos **no** deben copiarse a la imagen cuando se ejecuta `COPY . .`

Ejemplo .dockerignore

```
node_modules
dist
```

Excluir `node_modules` es importante por dos razones: evita copiar cientos de MB innecesarios (las dependencias ya se instalan dentro del contenedor con `npm install`) y previene errores, ya que algunos paquetes contienen binarios compilados para el sistema operativo del anfitrión que no funcionarían dentro de Alpine Linux.

4. ¿Qué es DockerHub?

DockerHub es el registro público de imágenes de Docker más utilizado. Cumple para las imágenes la misma función que GitHub cumple para el código: permite almacenarlas, versionarlas y compartirlas.

Ofrece dos tipos de contenido:

- **Imágenes oficiales:** mantenidas por Docker o por los desarrolladores del software. Se identifican por no tener usuario en el nombre (`mysql`, `nginx`, `node`, `python`). En este proyecto se consume la imagen oficial `mysql:8`.
- **Imágenes de usuario:** publicadas por cualquier persona con una cuenta. Siguen el formato `usuario/repositorio:etiqueta`, por ejemplo `tonelito762/pi-server:latest`.

4.1 Nomenclatura de las imágenes

```
tonelito762 / pi-server : latest
    |            |          |
  usuario    repositorio  etiqueta (tag)
```

La **etiqueta** (tag) identifica la versión. Si se omite, Docker asume `latest` de forma automática. Es buena práctica usar etiquetas de versión (`:1.0`, `:2.1`) en proyectos reales, ya que `latest` es ambigua y puede apuntar a contenido distinto con el tiempo.

4.2 Obtener imágenes de DockerHub

El comando `docker pull` descarga una imagen del registro hacia la máquina local:

```
# Forma general
docker pull <usuario>/<repositorio>:<tag>
```

```
# Descargar la imagen oficial de MySQL en su version 8
docker pull mysql:8

# Descargar las imagenes publicadas de este proyecto
docker pull tonelito762/pi-server:latest
docker pull tonelito762/pi-client:latest
```

No siempre es necesario ejecutar `pull` de forma explícita: cuando se usa `docker run` o `docker compose up` con una imagen que no existe localmente, Docker la descarga automáticamente.

4.3 Subir imágenes a DockerHub

El proceso consta de tres pasos: iniciar sesión, etiquetar la imagen con el nombre de usuario y hacer el push.

Paso 1 — Iniciar sesión:

```
docker login -u tonelito762
```

Solicitará la contraseña o, preferiblemente, un **Access Token** generado desde *Account Settings → Personal access tokens* en el sitio de DockerHub. Usar un token es más seguro porque puede revocarse en cualquier momento sin cambiar la contraseña de la cuenta.

Paso 2 — Etiquetar las imágenes:

DockerHub exige que el nombre de la imagen incluya el usuario propietario. Como Docker Compose genera nombres locales sin usuario (`docker-pi-server`), es necesario crear una etiqueta adicional:

```
# Forma general
docker tag <imagen-local> <usuario>/<repositorio>:<tag>

# Comandos usados en este proyecto
docker tag docker-pi-server tonelito762/pi-server:latest
docker tag docker-pi-client tonelito762/pi-client:latest
```

El comando `docker tag` **no duplica la imagen en disco**: crea un segundo nombre que apunta al mismo objeto. Por eso ambos nombres reportan el mismo tamaño.

Paso 3 — Subir las imágenes:

```
# Forma general
docker push <usuario>/<repositorio>:<tag>

# Comandos usados en este proyecto
docker push tonelito762/pi-server:latest
docker push tonelito762/pi-client:latest
```

4.4 Imágenes publicadas en este proyecto

Imagen	Enlace	Tamaño
`tonelito762/pi-server:latest`	https://hub.docker.com/r/tonelito762/pi-server	210 MB
`tonelito762/pi-client:latest`	https://hub.docker.com/r/tonelito762/pi-client	93 MB

Ambas son **públicas**, por lo que cualquier persona puede descargarlas sin necesidad de autenticarse.

Para cerrar la sesión al terminar de trabajar:

```
docker logout
```

5. Comandos esenciales de Docker

5.1 Construir imágenes

```
# Forma general
docker build -t <nombre-imagen> <ruta-del-contexto>

# Construir el servidor de este proyecto
docker build -t tonelito762/pi-server ./server

# Construir el cliente pasando una variable de build
docker build -t tonelito762/pi-client --build-arg
VITE_API_URL=http://localhost:4000 ./client
```

La opción `-t` (de *tag*) asigna un nombre a la imagen. La ruta final indica el **contexto de construcción**: la carpeta que Docker envía al motor y dentro de la cual busca el `Dockerfile`.

5.2 Listar imágenes y contenedores

```
# Listar todas las imagenes descargadas o construidas localmente
docker images

# Listar los contenedores que estan corriendo en este momento
docker ps

# Listar todos los contenedores, incluyendo los detenidos
docker ps -a
```

Ejemplo de salida real de `docker ps` en este proyecto:

```
NAMES                STATUS          PORTS
docker-pi-client-1   Up 13 seconds   0.0.0.0:3000->80/tcp
docker-pi-server-1   Up 13 seconds   0.0.0.0:4000->4000/tcp
docker-pi-db-1       Up 14 seconds   0.0.0.0:3306->3306/tcp
```

5.3 Ejecutar contenedores

```
# Forma general
docker run [opciones] <imagen>

# Levantar el servidor de este proyecto
docker run -d --name pi-server -p 4000:4000 tonelito762/pi-server

# Levantar el cliente de este proyecto
docker run -d --name pi-client -p 3000:80 tonelito762/pi-client
```

Opciones más utilizadas:

Opción	Significado
`-d`	*Detached*: ejecuta el contenedor en segundo plano.
`--name`	Asigna un nombre al contenedor (si se omite, Docker genera uno aleatorio).
`-p host:contenedor`	Publica un puerto del contenedor en la máquina anfitriona.
`-e VARIABLE=valor`	Define una variable de entorno dentro del contenedor.
`-v origen:destino`	Monta un volumen o carpeta dentro del contenedor.
`--network`	Conecta el contenedor a una red específica de Docker.
`--rm`	Elimina automáticamente el contenedor al detenerse.

Es fundamental comprender el mapeo de puertos `-p 3000:80`: el **primer número** es el puerto en la máquina anfitriona (el que se escribe en el navegador) y el **segundo** es el puerto dentro del contenedor.

5.4 Inspeccionar y depurar

```
# Ver los registros (logs) de un contenedor
docker logs docker-pi-server-1

# Seguir los logs en tiempo real
docker logs -f docker-pi-server-1

# Ver solo las ultimas 20 lineas
docker logs --tail 20 docker-pi-db-1

# Abrir una terminal interactiva dentro de un contenedor
docker exec -it docker-pi-server-1 sh

# Conectarse a MySQL dentro del contenedor de la base de datos
docker exec -it docker-pi-db-1 mysql -uroot -psecret piapp
```

Las opciones `-it` combinan `-i` (interactivo, mantiene la entrada abierta) y `-t` (asigna una terminal). Se usan siempre juntas para obtener una sesión utilizable.

Ejemplo de consulta directa a la base de datos para verificar que el CRUD funciona:

```
docker exec docker-pi-db-1 mysql -uroot -psecret piapp -e "SELECT * FROM users;"
```

5.5 Detener y eliminar

```
# Detener un contenedor en ejecucion
docker stop docker-pi-server-1

# Iniciar un contenedor previamente detenido
docker start docker-pi-server-1

# Reiniciar un contenedor
docker restart docker-pi-server-1

# Eliminar un contenedor (debe estar detenido)
docker rm docker-pi-server-1

# Eliminar una imagen
docker rmi tonelito762/pi-server:latest

# Eliminar recursos no utilizados (contenedores detenidos, redes, cache)
docker system prune
```

Conviene recordar la jerarquía: no se puede eliminar una imagen (`rmi`) si existe algún contenedor creado a partir de ella; primero hay que eliminar el contenedor (`rm`), y para ello debe estar detenido (`stop`).

6. Docker Compose

Cuando una aplicación se compone de varios contenedores que deben comunicarse entre sí, levantarlos manualmente uno por uno con `docker run` resulta tedioso y propenso a errores: hay que crear la red, recordar el orden de arranque y repetir todas las variables de entorno en cada ocasión.

Docker Compose resuelve este problema permitiendo describir toda la aplicación en un único archivo YAML y levantarla con un solo comando.

Con Docker Compose, todo lo anterior se reduce a:

```
docker compose up -d
```

6.2 El archivo docker-compose.yml

Archivo `docker-compose.yml` de ejemplo:

```yaml
services:
  db:
    image: mysql:8
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: piapp
    volumes:
      - db-data:/var/lib/mysql
      - ./db/init:/docker-entrypoint-initdb.d
    ports:
      - "3306:3306"

  server:
    build: ./server
    restart: unless-stopped
    depends_on:
      - db
    environment:
      DB_HOST: db
      DB_PORT: 3306
      DB_USER: root
      DB_PASSWORD: secret
      DB_NAME: piapp
      PORT: 4000
    ports:
      - "4000:4000"

  client:
    build:
      context: ./client
      args:
        VITE_API_URL: http://localhost:4000
    restart: unless-stopped
    depends_on:
      - server
    ports:
      - "3000:80"

volumes:
  db-data:
```

Elementos clave del archivo:

- **`services`** — cada entrada define un contenedor. El nombre del servicio (`db`, `server`, `client`) funciona además como **nombre de host** dentro de la red interna.
- **`image` vs `build`** — `image` consume una imagen ya existente (como `mysql:8`); `build` construye la imagen a partir de un `Dockerfile` local.
- **`environment`** — define las variables de entorno del contenedor. Nótese que `DB_PASSWORD` del servidor coincide con `MYSQL_ROOT_PASSWORD` de la base de datos.
- **`depends_on`** — controla el orden de arranque. Es importante saber que solo espera a que el contenedor **inicie**, no a que el servicio interno esté listo para recibir conexiones.
- **`volumes`** — el volumen nombrado `db-data` conserva los datos de MySQL aunque el contenedor se elimine; el montaje `./db/init` ejecuta automáticamente los scripts `.sql` la primera vez que se crea la base de datos.
- **`ports`** — publica los puertos hacia la máquina anfitriona.

6.3 Red interna y resolución por nombre

El aspecto más útil de Docker Compose es que crea automáticamente una red privada e incorpora en ella a los tres contenedores. Dentro de esa red, Docker ejecuta un servidor DNS interno que resuelve el **nombre de cada servicio** a la IP del contenedor correspondiente.

Gracias a esto, la configuración de conexión del servidor puede referirse a la base de datos simplemente como `db`:

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,   // vale "db", el nombre del servicio
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});
```

No es necesario conocer ni fijar direcciones IP: Docker se encarga de la resolución.

6.4 Comandos de Docker Compose

```
# Construir las imagenes y levantar todos los servicios en segundo plano
docker compose up --build -d

# Levantar sin reconstruir (usa las imagenes ya existentes)
docker compose up -d

# Ver el estado de los servicios
docker compose ps

# Ver los logs de todos los servicios
```

```
docker compose logs

# Ver los logs de un servicio especifico en tiempo real
docker compose logs -f server

# Detener y eliminar los contenedores y la red
docker compose down

# Detener y eliminar tambien los volumenes
docker compose down -v

# Reconstruir un solo servicio
docker compose build server
```

Es importante distinguir `docker compose down` de `docker compose down -v`: el primero conserva el volumen `db-data` y, por lo tanto, los usuarios registrados; el segundo elimina el volumen.

7. Resumen de comandos

Comando	Descripción
`docker build -t nombre ruta`	Construye una imagen a partir de un Dockerfile
`docker images`	Lista las imágenes locales
`docker run -d -p 3000:80 imagen`	Crea y ejecuta un contenedor
`docker ps` / `docker ps -a`	Lista contenedores activos / todos
`docker logs -f contenedor`	Muestra los registros en tiempo real
`docker exec -it contenedor sh`	Abre una terminal dentro del contenedor
`docker stop` / `start` / `restart`	Controla el ciclo de vida del contenedor
`docker rm` / `docker rmi`	Elimina contenedores / imágenes
`docker pull usuario/imagen`	Descarga una imagen desde DockerHub
`docker login -u usuario`	Inicia sesión en DockerHub
`docker tag origen usuario/destino`	Renombra una imagen para poder subirla
`docker push usuario/imagen`	Publica una imagen en DockerHub
`docker compose up --build -d`	Construye y levanta todos los servicios
`docker compose down`	Detiene y elimina los contenedores
`docker compose logs -f servicio`	Muestra los logs de un servicio
`docker system prune`	Libera espacio eliminando recursos sin usar

8. Referencias

- Documentación oficial de Docker — https://docs.docker.com/
- DockerHub — https://hub.docker.com/
- Referencia del Dockerfile — https://docs.docker.com/engine/reference/builder/
- Referencia de Docker Compose — https://docs.docker.com/compose/compose-file/